Security self-assessment

Security hardening is implemented, and the recorded security checks are complete.

This assessment explains what protects the working application, how those protections were tested and where their limits remain. Private privileged accounts require MFA. The public judge demo keeps its four documented passwords in a separate profile.

The October 20 checks passed locally and on hosted Windows/Linux. The local backend suite passed all 251 tests. Each hosted runner passed 250 and skipped one optional OCR model test. This is evidence for the local sandbox, not a penetration-test certificate or public-hosting approval.

Accounts and public passwords

Private setup generates random passwords and stores only Argon2 hashes in the database. The local credential file and signing secret stay outside Git and the source archive. Login uses short-lived signed tokens kept in browser memory and linked to revocable server sessions. Logout, password changes, recovery and authority changes revoke affected sessions. Requests reload the current active user, role and project membership.

The separate public demo intentionally publishes four fictional passwords. It uses a marked data/public-demo profile and localhost port 8003. It refuses unrelated data and conflicting accounts; it never resets existing private passwords. Its settings ignore private environment/.env values. Public passwords are not a secret or a suitable configuration for an Internet-facing app. Signing secrets are still generated privately. The launcher disables paid services and optional model access.

Roles and independent review

Role	Authority	Boundary
Operator	Prepare, upload, correct and analyze	Cannot approve/execute payments
Finance Manager	Confirm procurement, check estimates/controls and approve eligible payments	Independent work only; BDT 100,000 payment ceiling
Admin	Approve checked budgets/controls, manage access and eligible payments	Cannot approve retained work they prepared/checked; configured payment ceiling
Auditor	Read assigned records, receipts and integrity results	No writes

Retained contributors survive edits and revisions. Approvals bind exact snapshots and current sources. Execution rechecks preparer, approver and executor authority. Account/source changes invalidate affected pending proposals and release reservations. Archived runs remain read-only through ordinary business endpoints.

Controls and remaining risks

Risk	Implemented protection	Remaining boundary
Password theft/login abuse	Argon2, generic errors, normalized account/IP failure buckets, expiry and Retry-After	Persistent limits, offline one-use recovery and required private Admin/Finance MFA; no SSO or email recovery
Forged/stale token	Fixed signing policy, expiry, issuer/audience and current database access	Host compromise can expose signing material

Risk	Implemented protection	Remaining boundary
Untrusted document	Private files, explicit human confirmation, escaped text and image previews	Human review may be wrong; parser vulnerabilities remain possible
Beneficiary substitution	Source-linked supplier changes with separate checking/approval	Real identity/callback evidence needs people
Duplicate quantity/payment	Invoice identity, receipt allocations, payable bounds, one active proposal and unique settlement per proposal	False source records can still misstate physical delivery
Changed approval	Source/version/policy/content snapshot and current revalidation	A database owner can rewrite underlying state
Partial settlement failure	One cash/payment/ledger/allocation/receipt/audit transaction; rollback	No distributed bank transaction has been exercised
Cash shortfall	Current cash and base/delayed reserve checks	Expected inflows are assumptions
Prompt injection	Document text is data; deterministic decisions; no AI approval/payment tools	Optional explanation text may still be wrong
Resource abuse	Pre-parser body limits, bounded JSON/CSV/XLSX, PDF child time/page/pixel/output/memory/CPU limits	Not antivirus or a complete malware sandbox
Provider abuse	Fixed origins, bounded responses, stored task IDs, server-only keys, explicit paid enablement, atomic daily reservations and unknown-submission holds	Paid/live recovery needs separate verification
Spreadsheet injection	Reject imported formulas; escape exported formula-like cells	Review files in a suitable local environment
Browser disclosure	Memory-only tokens, protected previews/downloads and typed responses	XSS on an authenticated origin can access browser memory
Audit tampering	Canonical SHA-256 chain, serialized appends and first-failure verification	Full rewrite or tail removal needs an independent trusted checkpoint

Financial records and jobs

Estimates do not authorize procurement or payments. Cost and selling totals remain separate. Installments use one accepted quantity claim and exact cash reservations. Credits/corrections do not refund cash or release physical entitlement. Financial correction and accepted-cost reversal need separate linked reviews.

Jobs retain attempts, leases, heartbeats, bounded retry and restart recovery. Stale publishers are fenced. Unknown paid submissions require manual reconciliation. The supported deployment is one local process, not a distributed worker platform.

Logs contain request IDs and timing/status metadata. They omit credentials, tokens, provider keys and document contents. Numeric and upload bounds apply before expensive expansion/parsing. Hashing and parser work stay outside ordinary financial write locks, followed by current authority/version checks.

Evidence and deployment limits

See actual verification for tests, HTTP/browser workflows, parser checks and local workload results. Known-advisory scans describe published issues at their execution time; they do not prove complete security. Automated accessibility checks do not replace human assessment.

Recordings use isolated databases and addresses. Their provenance binds application source, duration and decoding. Packaging excludes private credentials, databases and uploads. The public sample-password fixture is intentionally included and labelled.

Before any wider deployment, independently review hosting, TLS, identity lifecycle, authenticator operations, OS isolation, malware handling, backups, external audit checkpoints, monitoring, data retention and incident response. See remaining work. No real bank transfer, professional standards certification or security certification is claimed.

Twenty-item security review

Release date: October 20, 2026. “Verified” below means local automated acceptance of the named control, not independent certification. Paid-provider and professional/deployment acceptance remain separate. The four README demo passwords are intentionally public and remain unchanged.

No.	Requested check	Status and implemented control	Evidence and practical limit
1	Keep API keys out of the frontend	Verified locally: credentials stay in backend settings; provider status exposes enabled/disabled state only	Secret scan checks frontend source, built assets and archives; private-profile secrets are compared without printing them
2	Exclude .env files	Verified locally: Git, Docker and source packaging exclude .env variants and private key/credential files; empty .env.example stays public	Public-folder filename check and release archive checks
3	Scan Git history	Verified: pinned Gitleaks 8.30.1 scans every available ref, current files/build and historical release archives, including nested contents	Redacted security-secret-scan.json; pattern scans do not prove absence of every possible secret
4	Rotate exposed keys	No confirmed exposure identified by the scans; no key rotation claimed	Exact local private-value comparisons also found no matches. Any future genuine leak requires provider revocation/replacement by its owner; Git history is preserved
5	Check permissions every request	Verified locally: session, account active state, current role and membership are checked on the server	Auth and project-isolation tests; no reliance on hidden UI controls
6	Distrust client IDs, roles and prices	Verified: database identity and policy control authority; monetary calculations and payment snapshots remain server-owned	Forged role/identity tests and financial regression scenarios; authorized users can still prepare explicitly reviewed estimates
7	Isolate users' data	Verified: unrelated projects, document details and previews are unavailable to an unassigned account	Two-account tests; authorized members deliberately share project business records

No.	Requested check	Status and implemented control	Evidence and practical limit
8	Protect database and files	Verified locally: private runtime permissions, safe storage paths and link/traversal rejection	Windows local checks; Linux permission checks run in hosted CI. OS administrators remain trusted; files are not encrypted
9	Protect direct Admin calls	Verified: direct administration and recovery calls require current Admin authority	Operator and unassigned-account requests are denied; last-Admin safeguards remain
10	Test login/logout/recovery	Verified locally: persistent 30-minute sessions, server logout, current-password change and hashed single-use 15-minute recovery tokens	Auth tests and browser security workflows. Public demo passwords cannot be changed; Admin recovery needs the local owner
11	Limit login/signup/expensive calls	Verified locally: persistent account/IP admission and failure buckets, operation limits and Retry-After	Concurrency, restart and boundary tests. No public signup exists; account creation is Admin-only
12	Cap AI/provider usage	Verified locally: paid providers disabled by default; positive limits and cost bounds required; reservations commit before dispatch	Mocked budget/retry tests. Local model has daily/concurrency/token/time/response bounds. Actual paid billing still needs separately authorized live verification
13	Validate inputs on server	Verified: typed contracts, bounded decimal/JSON/import parsing, explicit versions and business validation	Input and financial suites; front-end validation is only a convenience
14	Prevent SQL injection	Verified: ORM/bound parameters and allowlisted register behavior; login injection text grants no access	Query review and adversarial login/financial tests; maintenance SQL operates on controlled schema names
15	Prevent scripts from user content	Verified locally: escaped React/text content, raster document previews, attachment downloads and same-origin CSP with no inline scripts	Browser CSP test and upload tests. Inline styles remain allowed for charts; authenticated-origin XSS would still be serious
16	Limit uploads	Verified: file allowlists/signatures, 10 MB file bound, 50 PDF pages, pre-parser body bounds and bounded workers	Existing parser/upload tests; not an antivirus or complete malware sandbox
17	Verify payments before paid-feature access	Not applicable: this app has no subscription billing or paid-feature unlock endpoint	Sandbox project settlement is revalidated on the server. No real gateway/webhook or real payment verification is claimed
18	Disable debug and redact errors	Verified locally: debug=False, generic unexpected-error responses with request IDs and fixed metadata logging	Synthetic-secret error/log test and scanner output; no request bodies, tokens or provider URLs are logged

No.	Requested check	Status and implemented control	Evidence and practical limit
19	Back up and restore	Verified locally: consistent database plus referenced files, signing material and the separate MFA encryption key, checksummed private archive and isolated restore	Restore checks hashes, foreign keys and audit integrity; copied sessions/recovery tokens are revoked before fresh login. Never overwrites working data
20	Test two accounts and anonymous access	Verified: anonymous business endpoints reject access and separate accounts cannot cross project/file boundaries	Security tests plus role/payment browser workflows; public health, sign-in, recovery redemption and static UI are intentionally reachable

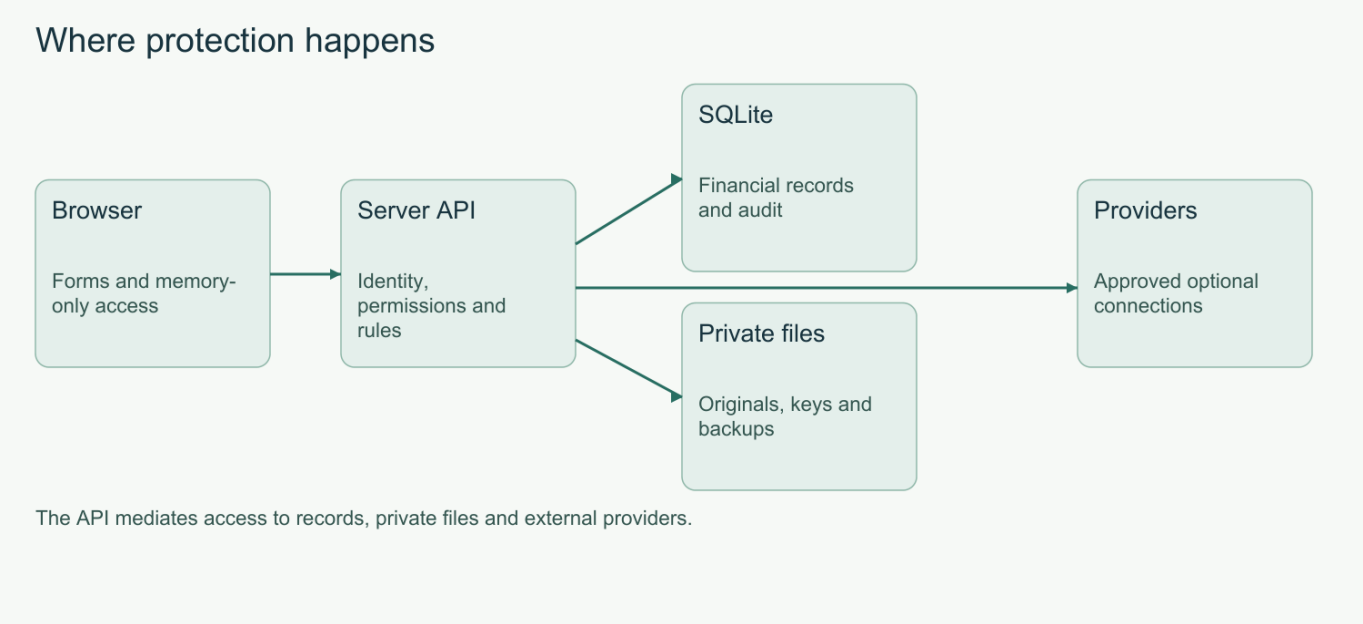
See account, recovery and backup instructions, actual release verification and the recorded test logs. The regression, security, hosted-platform and fresh-install checks are complete for the recorded release. Registration email was sent by the team; organizer confirmation and final project submission remain pending.

The 19-control security improvement checklist

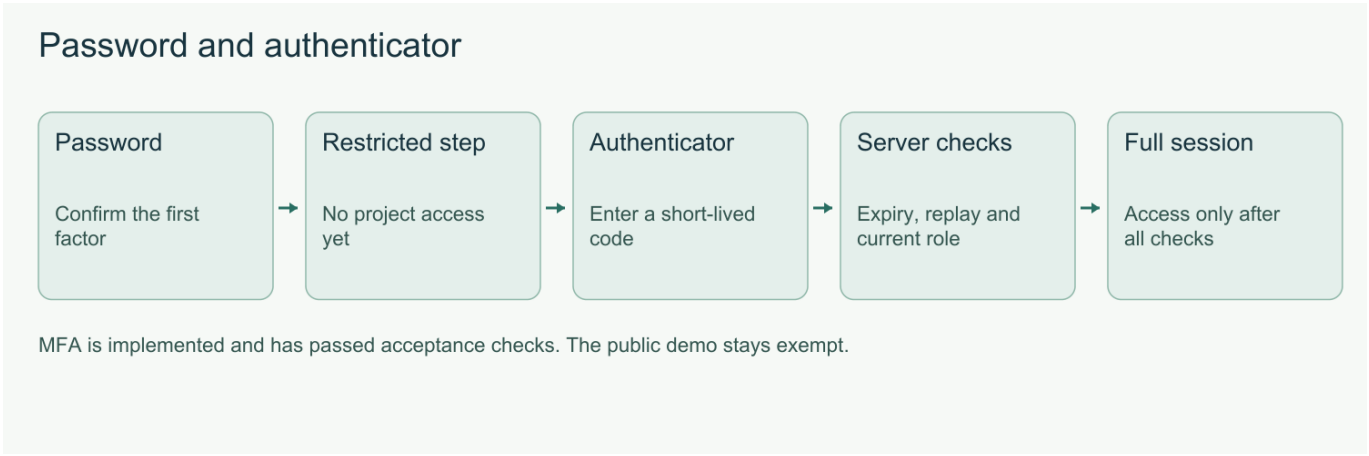
Security hardening is implemented. The recorded local security checks and the October 20 hosted Windows/Linux run passed. The local suite passed 251 backend tests, 10 frontend tests, 20 browser workflows and all eight original HTTP scenarios. Each hosted runner passed 250 backend tests and skipped one optional OCR model check. Read the actual hosted results.

Current labels: verified, deferred and not applicable. Seventeen controls are verified. Native database RLS is deferred and incoming webhooks are not applicable. The previous 20-item review remains separate.

Read the trust boundaries

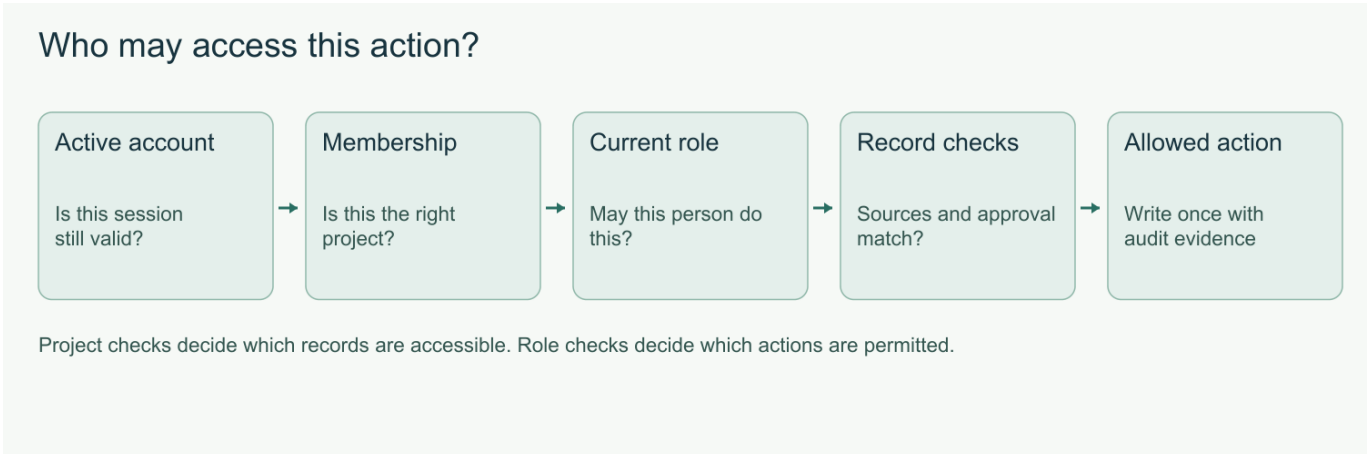


Where protection happens
The API mediates access to records, private files and external providers.



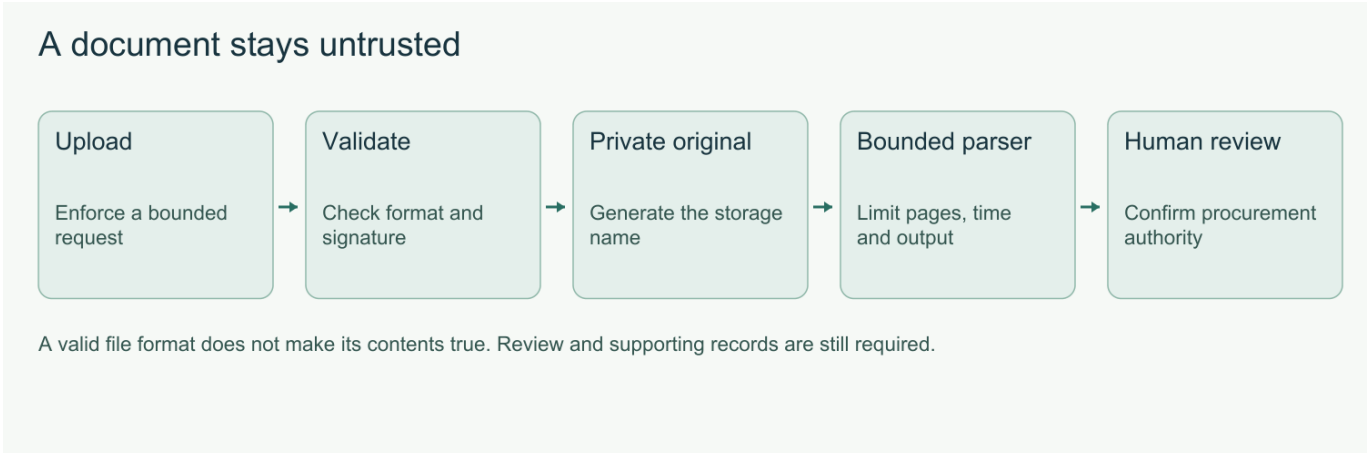
Password and authenticator

MFA is implemented and has passed acceptance checks. The public demo stays exempt.



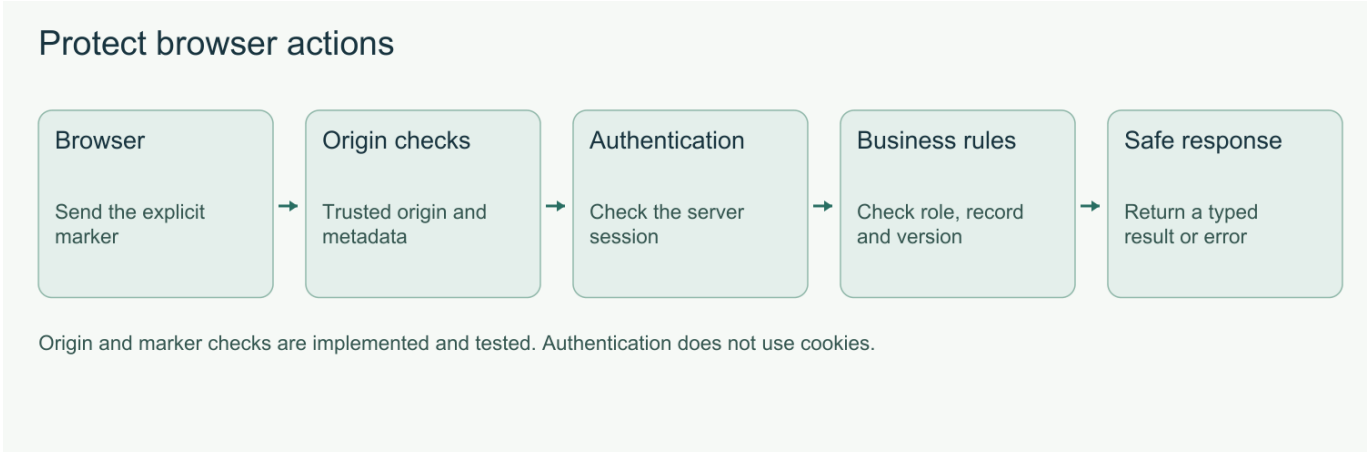
Who may access this action?

Project checks decide which records are accessible. Role checks decide which actions are permitted.



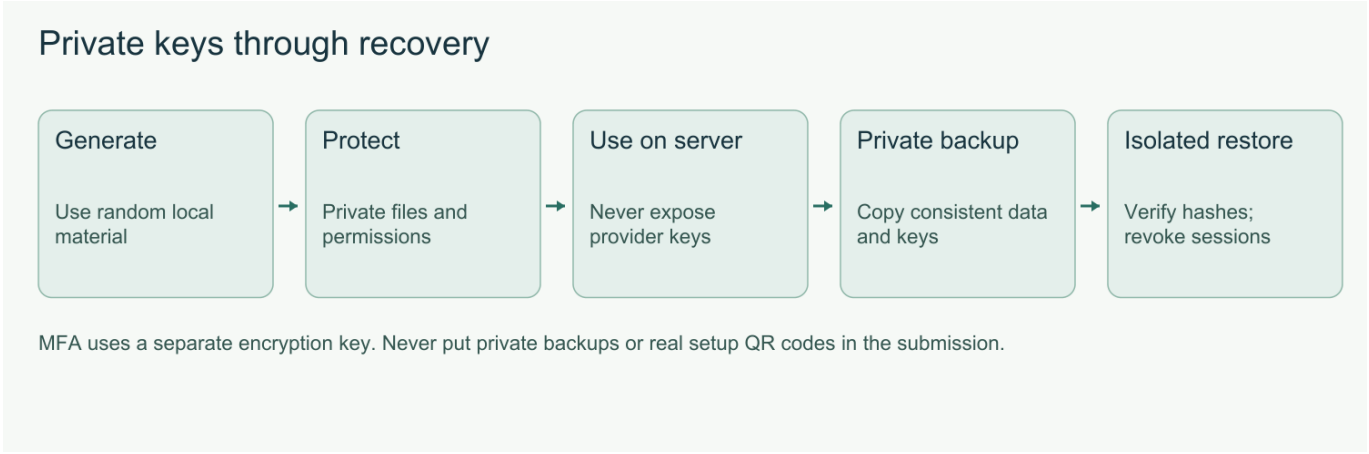
A document stays untrusted

A valid file format does not make its contents true. Review and supporting records are still required.



Protect browser actions

Origin and marker checks are implemented and tested. Authentication does not use cookies.



Private keys through recovery

MFA uses a separate encryption key. Never put private backups or real setup QR codes in the submission.

I01 - SQL injection

Purpose: Keep typed data separate from database commands.

Status: verified.

Current implementation: ORM predicates bind data; dynamic sorting and maintenance identifiers use fixed allowlists.

Evidence: Malicious login/register strings, literal apostrophes and invalid sorting tests; source query inventory.

Boundary: No independent penetration test; future raw SQL needs the same review.

I02 - Cross-site scripting (XSS)

Purpose: Stop saved text from becoming a browser script.

Status: verified.

Current implementation: React renders user text as text; links are validated, PDF previews are raster images and a restrictive CSP blocks scripts.

Evidence: Stored names, filenames, document descriptions and quotations; explicitly mocked malicious explanation response; upload and CSP tests.

Boundary: XSS on a trusted origin can still read memory; these cases do not prove all future views safe.

I03 - Cross-site request forgery (CSRF)

Purpose: Stop another website from submitting actions through a browser.

Status: verified.

Current implementation: Every mutating API call requires X-BuildTrust-Request: 1; browser Origin and Fetch Metadata are checked before business handling. No auth cookies.

Evidence: Missing marker, foreign origin and cross-site metadata rejection tests; valid browser and API workflows.

Boundary: An allowed-origin script remains trusted; a stolen bearer token is not prevented by CSRF controls.

I04 - File upload validation

Purpose: Accept bounded supported documents and retain their evidence.

Status: verified.

Current implementation: Signatures, allowed types, body/file size, archive expansion, pages, child deadlines and private paths are bounded. Procurement remains untrusted until confirmed.

Evidence: Disguised executable/PDF/JSON fixtures, parser bounds and protected preview tests.

Boundary: No antivirus or complete malware-sandbox claim; human evidence can still be false.

I05 - Server-side request forgery (SSRF)

Purpose: Keep outbound connections within intended destinations.

Status: verified.

Current implementation: Provider requests allow exact HTTPS hosts and ports, no credentials or redirects, and ignore environment proxies. Ollama is explicitly loopback-only. Listing URLs are never fetched.

Evidence: Disallowed schemes, hosts, ports and credentials tests; local-model normalization and mocked provider boundaries.

Boundary: Paid/live providers and a real local model retain separate external acceptance.

A01 - Object-level authorization

Purpose: Check which project owns every requested record.

Status: verified.

Current implementation: Current account and project membership are checked through shared resource helpers, including child records and protected files.

Evidence: Separate-project and anonymous API tests plus financial, archived-run and file browser workflows.

Boundary: This is application isolation, not native SQLite RLS or protection against an unrestricted DB owner.

A02 - Rate limiting

Purpose: Bound repeated sign-ins and expensive operations.

Status: verified.

Current implementation: Persistent account/IP admission limits and operation budgets include MFA verification and management. Challenges allow five failed codes.

Evidence: Restart/limit, five-attempt, expiry and concurrent usage tests; browser clients honor Retry-After.

Boundary: Shared local IPs may share admission limits; this is a single-process local release.

A03 - Password hashing

Purpose: Store a slow password hash instead of the password.

Status: verified.

Current implementation: Argon2 hashes protect private passwords. Password checks run outside ordinary financial writer locks.

Evidence: Login, recovery and hash-preservation tests; populated-copy migration comparison.

Boundary: Public-demo passwords are intentionally public; offline setup needs OS protection.

A04 - Multi-factor authentication (MFA)

Purpose: Require a password and an authenticator for privileged private users.

Status: verified.

Current implementation: Private Admin/Finance MFA is mandatory; optional roles may enroll. TOTP factors are encrypted, challenges/codes hashed, replay rejected and recovery use atomic. Only the owner resets lost factors.

Evidence: Enrollment, restricted access, replay, expiry, concurrent code use, role changes, reset and MFA backup/restore tests; real browser enrollment/sign-in.

Boundary: Local automated TOTP and browser evidence is not an independent phone/interoperability or phishing-resistance assessment. Public demo is exempt.

A05 - Server-side permissions

Purpose: Check whether the current role may perform the action.

Status: verified.

Current implementation: Every protected request reloads current account, role, membership and session assurance; financial execution revalidates sources and approval.

Evidence: Forged/weak-assurance sessions, current-role checks and original financial/permission regression tests.

Boundary: OS/database owner remains trusted. The copilot cannot approve or settle.

A06 - Native row-level security (RLS)

Purpose: Let the database itself enforce row-access policies.

Status: deferred.

Current implementation: SQLite uses server-side project predicates. Native database RLS has not been enabled.

Evidence: Architecture and project-isolation tests establish the implemented boundary only.

Boundary: Deferred: a future PostgreSQL migration needs native policies and acceptance.

S01 - JWT signing secrets

Purpose: Prevent forged or stale server sessions.

Status: verified.

Current implementation: Private random JWT keys must contain at least 32 UTF-8 bytes; existing strong keys are retained. Signed tokens also need an active server session and MFA assurance when applicable.

Evidence: Signing/session checks, old-session revocation and preserved-key migration/restore evidence.

Boundary: An unrestricted host owner can read signing material; TLS is required before any remote deployment.

S02 - Server-only API secrets

Purpose: Keep provider credentials outside the browser and public source.

Status: verified.

Current implementation: API credentials and separate MFA encryption keys stay in protected server storage and private backups; paid dispatch is off by default.

Evidence: Current/history/bundle/archive scanning, private-path/exclusion tests and restored-factor access.

Boundary: No genuine exposed provider key was found; unavailable provider ownership would still block a required rotation.

S03 - No persistent browser tokens

Purpose: Avoid leaving credentials in browser storage.

Status: verified.

Current implementation: Access tokens, restricted challenges, setup keys and recovery codes remain in memory. Only harmless view filters may persist.

Evidence: Browser storage inspection after MFA enrollment and frontend session tests.

Boundary: A page refresh requires sign-in; memory storage alone cannot defeat XSS.

C01 - Tight CORS and trusted origins

Purpose: Allow the intended local browser application only.

Status: verified.

Current implementation: Cross-origin access is disabled. Trusted hosts and browser origins are explicit per launcher; forwarded headers do not define trust.

Evidence: Foreign Origin/Host/Fetch Metadata rejection; actual isolated browser, recording and proxy configuration.

Boundary: A future hosted address needs deliberate configuration and HTTPS review.

C02 - Webhook signatures

Purpose: Authenticate incoming integration callbacks before processing them.

Status: not applicable.

Current implementation: No incoming webhook integration or real payment callback exists; callbacks remain disabled.

Evidence: Route inventory and sandbox architecture.

Boundary: Not applicable now. Any future webhook needs provider-specific signatures and replay controls.

C03 - Production source maps

Purpose: Do not publish browser debug maps with the running app.

Status: verified.

Current implementation: Production source maps are explicitly disabled. API/static handling rejects .map requests.

Evidence: Built-assets check finds no map files/directives; map-request regression.

Boundary: Public source is intentionally available under AGPL; hiding maps is not an access-control boundary.

C04 - Logs and lockfile hygiene

Purpose: Keep private values out of diagnostics and dependency records.

Status: verified.

Current implementation: Errors are generic with request IDs; structured logs omit tokens, passwords and content. Lockfiles and nested archives are included in redacted secret scans.

Evidence: Synthetic-marker log/error tests and pinned scanner output.

Boundary: Pattern scans cannot guarantee discovery of every possible secret format.

C05 - Dependency vulnerabilities

Purpose: Check published advisories and install reproducible versions.

Status: verified.

Current implementation: Reproducible locks include maintained MFA libraries with attribution. Known-advisory scans cover Python and frontend dependencies.

Evidence: Dated npm and pip-audit runs report no known advisories.

Boundary: New vulnerabilities can appear; this is not a complete supply-chain or penetration assessment.

Important distinctions

- XSS means content runs as a script. CSRF means a different website tries to submit an action. They require different controls.

- Project isolation is enforced by this application. Native database RLS remains deferred while SQLite is used.

- Incoming webhooks stay disabled. No real payment callback or signature-verification claim is made.

- Private Admin and Finance accounts require MFA; the four marked public-demo accounts keep their README passwords and simple login.

- Logs and dependency lockfiles must exclude private values. A dependency scan is not an independent security assessment.

Sources: OWASP CSRF, OWASP MFA, OWASP SSRF, PyOTP.